



Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Mestrado Integrado em Engenharia Informática

Unidade Curricular de Comunicação por Computadores

Ano Letivo de 2025/2026

Trabalho Prático 2 - PL6 -G9

Nuno Peixoto

A93244

Luís Ferreira

A98286

Salomé Faria

A108487

dezembro, 2025



Índice

1. Introdução	1
2. Arquitetura do Projeto	2
3. Implementação	3
3.1. MissionLink (UDP)	3
3.1.1. Acknowledgments	3
3.1.2. Janela	4
3.1.3. Fases Conexão	4
3.1.4. Retransmissão	4
3.2. TelemetryStream (TCP)	4
3.3. Ground Point (API)	6
3.3.1. Arquitetura e Integração	6
3.3.2. Endpoints Disponibilizados	7
3.3.2.1. /health	7
3.3.2.2. /missions	7
3.3.2.3. /rovers	7
3.3.2.4. /telemetry/latest	8
3.3.3. Formato das Mensagens e Consumo	8
3.3.4. Integração com o Ground Control	8
4. Testes e Resultados	9
5. Outros	12
5.1. Base de Dados	12
5.2. Logs	12
5.3. Packets (structs)	12
5.4. Telemetry (structs)	13
6. Dificuldades e Melhorias	14

1. Introdução

Este relatório documenta o trabalho desenvolvido no âmbito do Trabalho Prático nº2 da Unidade Curricular de Comunicações de Computadores, pertencente ao 3º ano da Licenciatura em Engenharia Informática, realizada no ano letivo 2025/2026, na Universidade do Minho.

Este trabalho consistiu no desenvolvimento de um sistema distribuído de comunicação para uma missão espacial simulada, no qual uma Nave-Mãe (*mothership*) em órbita que coordena e monitoriza múltiplos *rovers* na superfície de um planeta.

Para o desenvolvimento deste trabalho foi utilizada a linguagem de programação Python.

2. Arquitetura do Projeto

De modo a resolver o problema introduzido no enunciado do trabalho prático, foi desenvolvida a seguinte solução, seguindo o modelo cliente-servidor.

Para este modelo foi desenvolvido dois programas principais:

- Cliente (*rover*): Dispositivos localizados na superfície do planeta, que se conectam à *mothership*, cada *rover* é capaz de solicitar missões, executar missões simuladas (como exploração geográfica ou recolha de dados), e enviar continuamente telemetria sobre o seu estado e posição.
- Servidor (*mothership*): Componente central, localizado em órbita, responsável pela gestão da missão. A *mothership* atribui missões aos *rovers*, recebe confirmações e dados de monitorização (telemetrias) e expõe uma API de observação para monitorização em tempo real.

Para suportar a comunicação entre a *mothership* e os *rovers*, foram desenvolvidos dois protocolos aplicacionais com propósitos distintos.

- *MissionLink*: Utilizado para o envio de missões da *mothership* para os *rovers*, trata-se de um protocolo aplicacional construído sobre *UDP*, ao qual foram adicionados mecanismos de fiabilidade a nível aplicacional, como os do protocolo *TCP* (números de sequência, confirmações e retransmissões), garantindo uma comunicação crítica robusta.
- *TelemetryStream*: Utilizado para o envio contínuo de dados de telemetria dos *rovers* para a *mothership*, este protocolo assenta sobre o *TCP*, aproveitando as suas garantias de entrega e ordenação para uma transmissão fiável e estruturada de informações de monitorização.

Além da comunicação direta com os *rovers*, a *mothership* expõe uma API de Observação, que disponibiliza informação consolidada sobre missões, estado dos rovers e telemetria recente. Um nó adicional, designado Ground Control, consome esta API e apresenta os dados, permitindo à equipa da missão acompanhar em tempo real o andamento das operações.

3. Implementação

3.1. MissionLink (UDP)

O protocolo MissionLink permite o envio de missões da nave-mãe para o rover. É contruído sobre *UDP*, sendo, portanto, necessário adotar mecanismos que assegurem uma comunicação fiável na camada aplicacional, semelhantes aos de um protocolo *TCP*. Em termos de implementação, a lógica do protocolo está dividida em duas classes. A classe *MissionLink()* representa a lógica por detrás do socket, gerindo a receção e envio de mensagens. A classe *MissionLinkConnection()* representa as especificidades de cada conexão, como por exemplo os dados relativos ao *RTT* e à *Sliding Window*.

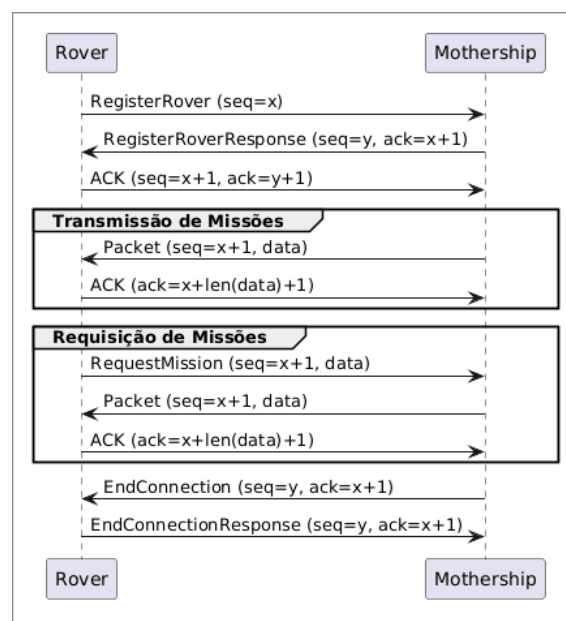


Figura 1: Diagrama de sequência de uma conexão MissionLink

3.1.1. Acknowledgments

Cada mensagem enviada aguarda confirmação de um pacote **ACK**, garantindo que os dados são devidamente recebidos. A cada ack recebido é ajustado o *RTT*, útil na retransmissão de pacotes.

3.1.2. Janela

A implementação de uma *Sliding Window* permite o controlo do fluxo de tráfego na rede, garantindo também que os pacotes enviados aguardam o ack correspondente.

Em termos de implementação, esta é constituída por uma lista de pacotes a enviar e por um dicionário de pacotes enviados que aguardam acknowledgment, cuja *key* é o número do ack esperado e o *value* o Pacote enviado.

O tamanho da janela define o número máximo de pacotes a ser enviados ao mesmo tempo, sempre um pacote é *acknowledged* é excluído do dicionário e é enviado um novo pacote em espera.

3.1.3. Fases Conexão

Assim como num protocolo TCP, dividimos a conexão em 3 fases (início de conexão, envio de missões, fim de conexão).

O início de conexão implementa o *Three-Way Handshake*. O rover começa por enviar um pacote **RegisterRequest** à nave-mãe, cujo endereço é conhecido, esta por sua vez atribui um id ao rover enviando-o num pacote **RegisterRequestResponse**. Por fim, o rover confirma a receção do mesmo através de um pacote **Ack**.

Estando a conexão estabelecida, o servidor envia pacotes com Missões, devidamente serializadas ao rover, aguardando o *acknowledgment* destas, tendo em conta o tamanho da janela, antes de enviar a próxima missão.

De forma a controlar a dispersão das missões, foi decidido que cada rover poderia ter no máximo 3 missões. Sempre que termina de realizar alguma, envia um pacote **Request-Mission** à nave-mãe.

A conexão é terminada pela nave-mãe, que em caso de já ter distribuído todas as missões, envia um pacote **EndConnection** ao rover, este por sua vez envia um pacote **EndConnectionResponse** (Fin + Ack) e termina a sua conexão com a nave-mãe.

3.1.4. Retransmissão

O mecanismo de retransmissão assegura uma comunicação fiável garantindo que não se percam pacotes. Cada conexão possui uma thread de retransmissão que verifica periodicamente os pacotes enviados que aguardam acknowledgment. Caso se tenha passado o tempo estabelecido no *RTT*, este é novamente enviado. Um pacote pode ser transmitido no máximo 5 vezes, após isso a nave-mãe assume que o rover não está mais disponível, terminando a conexão.

3.2. TelemetryStream (TCP)

O protocolo *TelemetryStream* tem como objetivo permitir a monitorização contínua do estado operacional dos rovers durante a execução das suas missões, sendo responsável por recolher, transmitir e processar dados de telemetria, garantindo que a *mothership* possua a informação atualizada sobre a posição, a bateria, a velocidade e o progresso das missões atribuídas a cada rover ao longo do tempo.

A arquitetura adotada segue um modelo cliente-servidor, onde cada *rover* atua como um cliente *TCP*, implementado através da classe *TelemetryStreamClient*, enquanto a *mothership* atua como um servidor *TCP*, através da classe *TelemetryStreamServer*. A comunicação é unidirecional, sendo os dados de telemetria enviados continuamente do *rover* para o servidor, permitindo à *mothership* acompanhar o estado de cada unidade de forma centralizada.

Cada missão é executada sequencialmente, para cada missão, o *rover* gera um objeto de telemetria inicial apenas uma vez, contendo uma posição inicial aleatória, nível de bateria máximo, velocidade aleatória dentro de limites definidos e estado operacional *DOING*. A partir desse momento, o mesmo objeto de telemetria é atualizado ciclicamente ao longo da execução da missão, refletindo as alterações no estado do rover.

Durante a execução da missão, o nível da bateria do *rover* decrementa, com base num valor fixo, para simular o consumo energético do *rover*, quando este nível de bateria alcança um certo intervalo, o *rover* muda para um estado de carregamento da bateria, neste caso, o seu estado operacional é alterado para *CHARGING* a velocidade do *rover* é definida como zero e a sua posição mantém-se constante. Durante o carregamento, a bateria é incrementada gradualmente até que seja atingido um nível seguro para retomar a missão (30%), ao longo deste processo, são enviadas telemetrias que refletem este novo estado energético.

Em qualquer estado operacional do *rover*, (*DOING*, *CHARGING*, *FINISHED*) todas as informações relevantes são encapsuladas num objeto da classe *Telemetry* que contém todos os dados sobre do rover sobre a execução de uma dada missão. Antes do envio de uma telemetria esta é serializada, permitindo a sua transmissão através da ligação *TCP* estabelecida com o *TelemetryStreamServer* ao *TelemetryServer*, sobre a função “send_current_telemetry”.

No lado do servidor, o *TelemetryStreamServer* é responsável por aceitar múltiplas ligações simultâneas de *rovers*, criando uma *thread* dedicada para cada cliente conectado. O servidor lê os dados recebidos utilizando um protocolo simples baseado no envio prévio do tamanho da mensagem, garantindo a leitura completa de cada pacote de telemetria. Após a desserialização, a telemetria é armazenada em memória como o estado mais recente de cada *rover* e é também persistida numa base de dados.

O diagrama de sequência ilustra o protocolo de comunicação, *TelemetryStream*, estabelecido entre um rover e a mothership para transmissão de dados de telemetria.

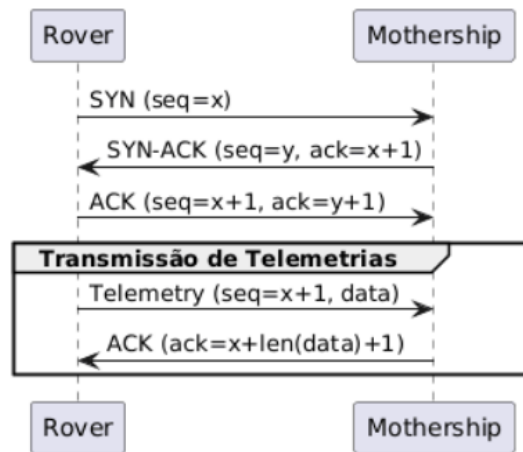


Figura 2: Diagrama de sequência de uma conexão TelemetryStream

3.3. Ground Point (API)

O **Ground Point** corresponde à componente responsável por disponibilizar uma **API de Observação**, permitindo a monitorização do estado global do sistema por entidades externas, nomeadamente o nó **Ground Control**. Esta API funciona como um ponto de acesso centralizado à informação mantida pela **Mothership**, possibilitando a consulta, em tempo real (ou próximo do tempo real), do progresso das missões e do estado dos rovers em operação.

A API foi implementada seguindo o paradigma **REST over HTTP**, recorrendo à framework **Flask**, escolhida pela sua simplicidade e por permitir construir rapidamente serviços web leves. Esta camada de observação encontra-se desacoplada dos protocolos de comunicação de baixo nível (MissionLink e TelemetryStream), permitindo observar o sistema sem interferir com a distribuição de missões (UDP) nem com a recolha contínua de telemetria (TCP).

3.3.1. Arquitetura e Integração

A API de Observação executa no lado da **Mothership** e integra-se com dois componentes principais:

- **Base de Dados (SQLite)**: armazena de forma persistente a lista de missões carregadas a partir do ficheiro `missions.json`. A API utiliza esta base de dados para disponibilizar informação consistente sobre as missões, mesmo após reinícios do sistema.
- **TelemetryStreamServer**: mantém em memória a telemetria mais recente recebida de cada rover (estrutura `telemetry_data` protegida por **lock**). Esta informação é utilizada pela API para fornecer uma visão imediata do estado operacional dos rovers.

A API é inicializada num **thread** dedicado, de forma a correr concorrentemente com:

- o servidor MissionLink (UDP), responsável por registar rovers e atribuir missões;
- o servidor TelemetryStream (TCP), responsável por receber telemetria periódica.

Esta abordagem evita bloqueios no funcionamento principal e mantém a camada de observação independente da execução das restantes tarefas.

3.3.2. Endpoints Disponibilizados

A API disponibiliza os seguintes endpoints:

3.3.2.1. /health

Endpoint de verificação rápida de disponibilidade. É utilizado para testes e para validação inicial por parte do **Ground Control**.

3.3.2.2. /missions

Devolve a lista completa de missões conhecidas pela **Mothership**. A informação é lida diretamente da base de dados (tabela missions), incluindo parâmetros estáticos (área, tarefa, duração, intervalo de reporte) e o estado atual associado a cada missão.

Cada missão inclui:

- Identificador da missão (mission_id);
- Área geográfica (area, coordenadas de início e fim);
- Tarefa (task);
- Duração prevista (duration);
- Intervalo de reporte (report_time);
- Estado (state).

Este endpoint permite ao **Ground Control** apresentar uma visão global do plano de missões e acompanhar a sua evolução ao longo do tempo.

3.3.2.3. /rovers

Disponibiliza a lista de rovers considerados ativos (rovers que já enviaram telemetria). A informação provém da estrutura telemetry_data mantida pelo **TelemetryStreamServer**.

Para cada rover, a API devolve:

- Identificador (rover_id);
- Estado operacional (status, derivado do operational_status da telemetria);
- Última telemetria recebida (last_telemetry), contendo:
 - posição (position);
 - nível de bateria (battery_level);
 - velocidade (velocity);
 - estado operacional (operational_status);
 - missão associada (mission_id), quando disponível.

A inclusão de mission_id na telemetria mais recente é particularmente relevante, pois permite ao **Ground Control** indicar diretamente qual missão cada rover está a executar.

3.3.2.4. /telemetry/latest

Permite obter a última telemetria recebida de cada rover, organizada por identificador de rover. A informação é extraída diretamente de `TelemetryStreamServer.telemetry_data`, garantindo acesso imediato ao estado mais recente de cada rover.

3.3.3. Formato das Mensagens e Consumo

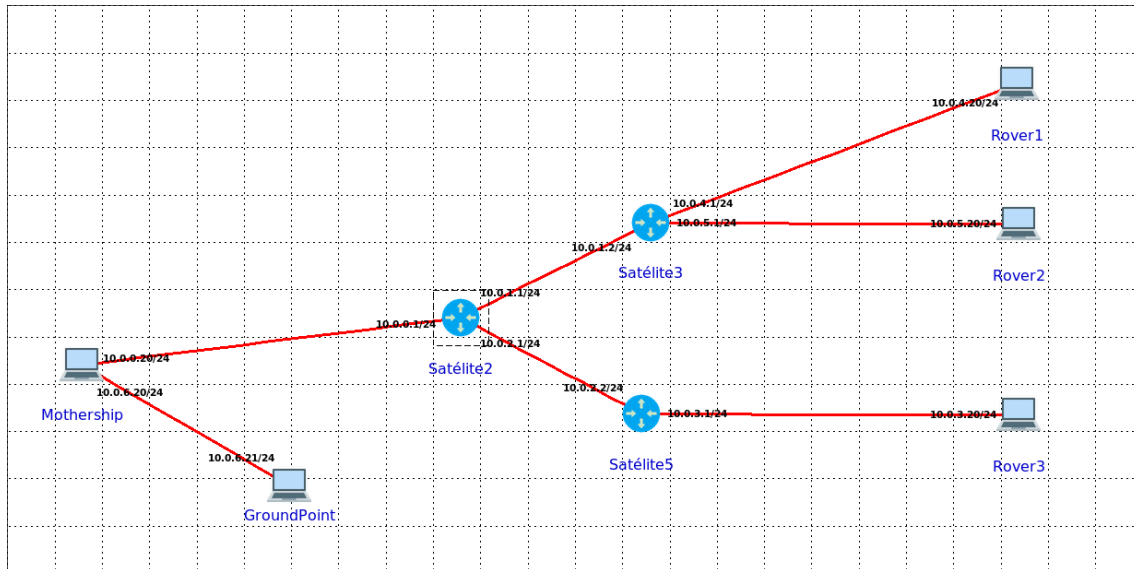
Todas as respostas da API são devolvidas no formato JSON, devido à sua simplicidade, legibilidade e compatibilidade com múltiplas linguagens e clientes. A API foi desenhada para suportar a verificação periódica por parte do **Ground Control**, que efetua chamadas regulares para obter atualizações do sistema.

3.3.4. Integração com o Ground Control

O **Ground Control** atua como cliente da API, consultando periodicamente endpoints como `/missions` e `/rovers` para construir uma vista de monitorização em terminal. Para reduzir o volume de informação apresentada, é mantido um estado local que permite detetar alterações (**diff**) entre ciclos consecutivos, apresentando apenas mudanças relevantes, como alterações no estado das missões, entrada ou saída de rovers ativos e variações significativas na telemetria.

4. Testes e Resultados

Para a realização de testes, foi desenvolvida a seguinte topologia:



A **Figura 3** demonstra a conexão do protocolo MissionLink entre nave-mãe e rover. A nave-mãe começa por inicializar a base de dados e inserir as missões passadas como argumento na mesma. De seguida, abre a conexão MissionLink e fica à espera por algum rover que se queira conectar. Assim que recebe um pedido de conexão por parte de um rover, começa o processo descrito no ponto [3.1.2 Fases Conexão](#)

```
root@Mothership:/volume/CC# python -m mothership-server missions.json mothership-server/mot  
hership.db  
[2025-11-13 16:51:47] [INFO] Missions table and database started successfully.  
[2025-11-13 16:51:47] [INFO] Mission M-001 inserted successfully  
[2025-11-13 16:51:47] [INFO] Mission M-002 inserted successfully  
[2025-11-13 16:51:47] [INFO] Mission M-003 inserted successfully  
[2025-11-13 16:51:47] [INFO] MissionLink server started on 0.0.0.0:9999.  
[2025-11-13 16:51:48] [INFO] Server confirms connection with 10.0.4.20:57394  
[2025-11-13 16:51:48] [INFO] Sending Seq=4011717113 to 10.0.4.20:57394  
[2025-11-13 16:51:48] [INFO] Received Ack=4011717114 from 10.0.4.20:57394  
□
```

```
root@Rover1:/volume/CC# python -m rover 10.0.0.20  
[2025-11-13 16:51:48] [INFO] MissionLink server started on 0.0.0.0:0.  
[2025-11-13 16:51:48] [INFO] Rover 0.0.0.0:0: requesting connection  
[2025-11-13 16:51:48] [INFO] Sending Ack=4011717112 to 10.0.0.20:9999  
[2025-11-13 16:51:48] [INFO] Sending Ack=4011717114 to 10.0.0.20:9999  
□
```

Figura 3: Protocolo MissionLink

Na **Figura 4** é usada uma bandwidth de 600bps, prolongando a latência no envio de pacotes. É possível verificar a retransmissão de pacotes e o reajuste do RTT.

```
root@Mothership:/volume/CC# python -m mothership-server missions.json mothership-server/mot
hershship.db
[2025-11-13 16:54:08] [INFO] Missions table and database started successfully.
[2025-11-13 16:54:08] [INFO] Mission M-001 inserted successfully
[2025-11-13 16:54:08] [INFO] Mission M-002 inserted successfully
[2025-11-13 16:54:08] [INFO] Mission M-003 inserted successfully
[2025-11-13 16:54:08] [INFO] MissionLink server started on 0.0.0.0:9999.
[2025-11-13 16:54:14] [INFO] Server confirms connection with 10.0.4.20:40947
[RETRANSMIT] Packet Seq=1737745687 to ('10.0.4.20', 40947), timeout=5.000s
[2025-11-13 16:54:20] [INFO] Sending Seq=1737745689 to 10.0.4.20:40947
[RETRANSMIT] Packet Seq=1737745689 to ('10.0.4.20', 40947), timeout=3.389s
[RETRANSMIT] Packet Seq=1737745689 to ('10.0.4.20', 40947), timeout=3.954s
[2025-11-13 16:54:30] [INFO] Received Ack=1737745688 from 10.0.4.20:40947
[RETRANSMIT] Packet Seq=1737745689 to ('10.0.4.20', 40947), timeout=4.801s
[2025-11-13 16:54:35] [INFO] Received Ack=1737745690 from 10.0.4.20:40947
[2025-11-13 16:54:51] [INFO] Received Ack=1737745690 from 10.0.4.20:40947
[2025-11-13 16:54:56] [INFO] Received Ack=1737745690 from 10.0.4.20:40947
[2025-11-13 16:55:07] [INFO] Received Ack=1737745690 from 10.0.4.20:40947
```

```
root@Rover1:/volume/CC# python -m rover 10.0.0.20
[2025-11-13 16:54:10] [INFO] MissionLink server started on 0.0.0.0:0.
[2025-11-13 16:54:10] [INFO] Rover 0.0.0.0:0: requesting connection
[2025-11-13 16:54:17] [INFO] Sending Ack=1737745688 to 10.0.0.20:9999
[2025-11-13 16:54:25] [INFO] Sending Ack=1737745688 to 10.0.0.20:9999
[2025-11-13 16:54:29] [INFO] Sending Ack=1737745690 to 10.0.0.20:9999
[2025-11-13 16:54:41] [INFO] Sending Ack=1737745690 to 10.0.0.20:9999
[2025-11-13 16:54:44] [INFO] Sending Ack=1737745690 to 10.0.0.20:9999
[2025-11-13 16:54:52] [INFO] Sending Ack=1737745690 to 10.0.0.20:9999
```

Figura 4: Retransmissão de Pacotes

5. Outros

5.1. Base de Dados

A Base de Dados relacional implementada armazena todas as missões inicialmente enviadas à nave-mãe como argumento no momento da sua inicialização, bem como as telemetrias recebidas pelos diferentes rovers.

Esta é manipulada através de queries SQL, usando a biblioteca de *python* *sqlite3*.

5.2. Logs

Com o objetivo de facilitar a visualização do funcionamento do programa, foi implementada uma função de logging. Desta forma, todas as ações da aplicação são registadas no terminal, acompanhadas do respetivo timestamp da sua ocorrência.

5.3. Packets (structs)

Cada pacote enviado é um extensão da classe ***Packet***, cujas variáveis de instância base são o número de sequência do pacote, o número do ack correspondente e o valor do enum ***PacketType***, necessário para a desserialização. Para cada classe são ainda definidos os métodos de serialização e desserialização adequados.

Packet			
Campo	Tipo de dados	Tamanho (bytes)	Descrição
Packet Type	u8	1	Identificador do tipo de pacote, definido pelo enum <i>PacketType</i>
Sequence Number	u32	4	Número de sequência do pacote
Acknowledgment Number	u32	4	Número de confirmação do pacote
Body	Bytes	Variável	Corpo do pacote, cujo conteúdo depende do <i>PacketType</i>

Tabela 1: Estrutura de um segmento **Packet**

5.4. Telemetry (structs)

Cada telemetria é enviada através da classe *Telemetry*, cujas variáveis de instância são o *ID* do *rover*, a sua posição, o seu nível de bateria, a sua velocidade, o *ID* da missão a ser executada e o nível operacional do *rover*.

Telemetry			
Campo	Tipo de dados	Tamanho (bytes)	Descrição
Rover ID	str	Variável	Identificador único do <i>rover</i>
Mission ID	str	Variável	Identificador da missão atualmente em execução
Position X	float	4	Coordenada X da posição atual do <i>rover</i> no espaço da missão
Position Y	float	4	Coordenada Y da posição atual do <i>rover</i> no espaço da missão
Position Z	float	4	Coordenada Z da posição atual do <i>rover</i>
Battery Level	float	4	Porcentagem de bateria do <i>rover</i>
Velocity	float	4	Velocidade atual do <i>rover</i>
Operational Status	u8	1	Estado operacional do <i>rover</i> , codificado segundo o <i>enum OperationalStatus</i>

Tabela 1: Estrutura de um segmento **Telemetry**

6. Dificuldades e Melhorias

Uma das principais dificuldades encontradas foi a implementação de mecanismos da camada de transporte na camada aplicacional, conciliando conhecimentos de redes com a complexidade dos sistemas distribuídos, de forma a garantir que o servidor fosse capaz de responder de maneira concorrente aos diversos rovers.

Um dos pontos a melhorar no trabalho é a thread-safety, apesar de algumas preocupações de exclusão mútua através do uso de locks, há certamente abertura para melhorar o projeto, tornando-o um sistema distribuído mais robusto. O uso de condições, sinais e eventos poderia também ser melhor explorado, tornando o programa mais dinâmico, por exemplo, através de triggers que verifiquem periodicamente o estado das conexões. Outro aspecto a implementar seria uma janela de congestão dinâmica.